

RefactorTactics — Product Requirements Document

Executive summary

RefactorTactics è un gioco tattico competitivo a turni simultanei, sviluppato con Unreal Engine, nel quale ogni giocatore pianifica in segreto rispetto alla squadra avversaria ma condivide in tempo reale le proprie intenzioni con gli alleati. Il risultato è un'esperienza basata su previsione, coordinazione e lettura del campo, non sulla velocità di esecuzione.

La principale differenza rispetto ai riferimenti del genere è che la mappa non è uno sfondo passivo. È un sistema di gioco vero e proprio: multilivello, modificabile, dotato di celle con proprietà, effetti, interazioni e collegamenti speciali. Terreno, altezza, coperture, visibilità, incendi, acqua, elettricità, ascensori, ponti, porte e portali partecipano direttamente alla risoluzione del turno.

Il secondo elemento distintivo è la collaborazione durante la pianificazione. Gli alleati vedono percorsi, bersagli, aree d'effetto, intenzioni, modifiche e stato di conferma degli altri membri del team. I nemici non ricevono questi dati, nemmeno a livello di replica di rete. Il sistema deve quindi essere progettato con una separazione rigorosa tra informazioni private, informazioni di squadra e stato pubblico.

Il terzo elemento è la personalizzazione orizzontale dei personaggi. Ogni personaggio mantiene una propria identità visiva e meccanica, ma può equipaggiare moduli, talenti, gadget, specializzazioni e varianti delle abilità. La progressione non deve concedere vantaggi statistici permanenti nel PvP: deve ampliare le possibilità strategiche senza creare giocatori numericamente superiori.

Il quarto elemento è l'estensibilità. Il supporto alle mod sarà l'ultima grande feature di prodotto, ma l'architettura dati dovrà essere predisposta sin dall'inizio. Terreni, abilità, personaggi, effetti ambientali e modalità dovranno essere definiti tramite asset, tag e contratti versionati, evitando dipendenze rigide da classi native.

La baseline di prodotto proposta è:

Area	Decisione di prodotto
Genere	Tattico competitivo a turni simultanei
Piattaforma iniziale	PC, tastiera e mouse; controller successivo
Squadre	3 contro 3 nel formato principale
Modalità iniziale	Controllo di obiettivi dinamici
Durata obiettivo	20–30 minuti per partita
Pianificazione	30 secondi, configurabile per modalità
Risoluzione	6–12 secondi per turno

Area	Decisione di prodotto
Visuale	Isometrica 3D con rotazione, zoom e gestione dei livelli
Monetizzazione prevista	Non definita nel presente PRD
Progressione	Orizzontale e cosmetica
Server	Autorevole, preferibilmente dedicato
Mod	Data-only inizialmente; niente codice nativo non fidato
Motore	Unreal Engine 5, versione stabile bloccata per milestone

Unreal offre sia C++ sia Blueprints come strumenti di gameplay: Epic raccomanda generalmente una base C++ che esponga API controllate, estendibili tramite Blueprint. C++ offre maggiore controllo su prestazioni, serializzazione, rete e merge; Blueprint accelera prototipazione e iterazione. Questo modello ibrido è particolarmente adatto a RefactorTactics. ¹

Il Gameplay Ability System, disponibile come plugin, fornisce astrazioni per abilità, attributi, costi, cooldown, effetti, tag e networking. È quindi un buon fondamento per i personaggi, ma non deve controllare direttamente il simulatore tattico: il simulatore resta il sistema autorevole che decide ordine e risultato delle azioni. ²

Decisione go/no-go: il progetto deve dimostrare entro il vertical slice che la combinazione di mappa multilivello, turni simultanei e intenti condivisi è leggibile e divertente. Modding avanzato, progressione estesa, matchmaking competitivo e grande quantità di contenuti non devono essere sviluppati prima di questa validazione.

Visione, obiettivi e perimetro

Visione del prodotto. RefactorTactics deve far sentire il giocatore come parte di una squadra che sta costruendo un piano sotto pressione. Il momento centrale non è soltanto l'esecuzione dell'abilità, ma la negoziazione visiva del piano: "io mi muovo qui, tu blocca quella scala, il terzo compagno elettrifica l'acqua".

La mappa deve produrre decisioni continuamente. La stessa abilità deve poter avere valore diverso a seconda della quota, del materiale della cella, della presenza di acqua, della copertura, del vento, della visibilità e delle connessioni verticali.

Pubblico di riferimento. Il pubblico principale comprende giocatori interessati a tattica competitiva, hero-based combat, deckbuilding leggero, giochi a turni e coordinazione cooperativa. Il prodotto deve essere comprensibile senza chat vocale, ma abbastanza profondo da premiare squadre organizzate.

Obiettivi di esperienza.

Obiettivo	Comportamento atteso
Coordinazione immediata	Un giocatore comprende il piano del team osservando la mappa per pochi secondi

Obiettivo	Comportamento atteso
Profondità senza build obbligatorie	Più configurazioni risultano valide per ogni personaggio
Mappa memorabile	I giocatori nominano e riconoscono aree, transizioni ed eventi della mappa
Informazione affidabile	Preview, warning e risoluzione producono risultati coerenti
Apprendimento progressivo	Il tutorial introduce una variabile tattica alla volta
Rigiocabilità	Personaggi, moduli e condizioni della mappa generano piani differenti
Estensibilità	Nuovi contenuti richiedono prevalentemente dati e asset, non modifiche al core

Non-obiettivi iniziali. Il primo rilascio non deve includere open world, campagna narrativa completa, combattimento in tempo reale, fisica distruttiva generalizzata, generazione procedurale delle mappe competitive, mercato interno, supporto console simultaneo o scripting nativo arbitrario per le mod.

Principi inderogabili.

Pilastro	Requisito
Mappa come attore	Le caratteristiche ambientali devono influire su movimento, tiro, visibilità, abilità e obiettivi
Turni simultanei	Tutti pianificano nello stesso intervallo e il server risolve un unico snapshot
Intenti condivisi	Gli alleati vedono i piani in tempo reale; gli avversari non ricevono quei dati
Personaggi modulari	La build modifica stile e funzione, non soltanto valori numerici
Mod support	I dati di gioco sono identificabili, versionabili e validabili
Determinismo osservabile	Lo stesso snapshot, regole, versione e seed devono produrre lo stesso risultato logico
Server authority	Il client propone azioni; il server valida e applica lo stato ufficiale

Formato della partita. La modalità principale, provvisoriamente denominata **Convergenza**, utilizza due o tre obiettivi ambientali per mappa. Le squadre guadagnano punti controllando, attivando o modificando tali obiettivi. Le eliminazioni contribuiscono al vantaggio ma non sono l'unica condizione di vittoria.

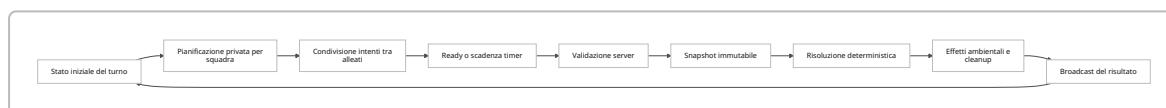
La configurazione iniziale proposta è:

Parametro	Target
Giocatori	3 contro 3

Parametro	Target
Turni massimi	12
Tempo di pianificazione	30 secondi
Tempo di grazia rete	2 secondi
Punti vittoria	8
Punti per obiettivo	1-2 in base alla difficoltà
Punti per KO	1
Rientro dopo KO	Dopo un turno completo
Abbandono	Bot o controllo condiviso, da definire in alpha

Il 3 contro 3 riduce il carico visivo e cognitivo prodotto da percorsi, aree, ping e interazioni ambientali. L'architettura deve comunque supportare 4 contro 4 per modalità personalizzate e playtest.

Ciclo fondamentale.



Il client deve mostrare una simulazione previsionale, non una promessa assoluta. La UI distingue quindi tre livelli:

Livello	Significato
Confermato	Il risultato non dipende da azioni nemiche sconosciute
Previsto	Risultato probabile in base allo stato pubblico corrente
Incerto	Può cambiare per collisioni, reazioni, interruzioni o azioni nemiche

La risoluzione segue una sequenza stabile:

1. snapshot degli intenti e del seed;
2. effetti di inizio turno e reazioni preparatorie;
3. transizioni e movimenti, risolti per micro-step;
4. controllo, difesa e interruzioni;
5. attacchi e abilità;
6. propagazione degli effetti ambientali;
7. KO, obiettivi, cooldown e cleanup.

L'ordine esatto deve essere data-driven, ma non modificabile liberamente nelle partite classificate.

Requisiti funzionali del gioco

Architettura della mappa. La mappa è un grafo tattico tridimensionale. Ogni nodo rappresenta una posizione valida; ogni arco rappresenta una transizione. La posizione logica non deve coincidere necessariamente con una griglia visiva perfettamente regolare.

```
FCellId
├─ X
├─ Y
├─ Layer
└─ LocalIndex
```

Il campo `Layer` distingue ponte, piano terra, tetto, tunnel o altri livelli sovrapposti. Gli archi verticali descrivono scale, rampe, ascensori, salti, zipline, portali e cadute.

Ogni cella contiene dati compatti, non un Actor completo. Gli Actor e gli ActorComponent sono adatti a oggetti e comportamenti riutilizzabili; per migliaia di celle è preferibile un archivio dati centralizzato, suddiviso in chunk, con Actor dedicati soltanto a rendering, collisione e interazioni visibili. Questa è una decisione architetturale del progetto, coerente con la distinzione Unreal tra Actor e componenti riutilizzabili. ³

Dato della cella	Esempi
Identità	ID stabile, coordinate, layer
Geometria tattica	Quota, pendenza, altezza disponibile
Superficie	Metallo, roccia, acqua, legno, vetro
Movimento	Costo base, blocco, direzioni consentite
Difesa	Copertura bassa, alta, direzionale
Visibilità	Opacità, fumo, illuminazione
Stato ambientale	Fuoco, ghiaccio, elettricità, veleno
Occupazione	Unità, oggetti, ostacoli temporanei
Interazione	Porta, terminale, leva, ascensore
Tag	<code>Terrain.Water</code> , <code>Height.High</code> , <code>Hazard.Fire</code>
Revisione	Versione locale per cache e pathfinding

I Gameplay Tags di Unreal sono etichette gerarchiche leggere pensate per identificare, categorizzare, confrontare e filtrare oggetti. Sono adatti a descrivere terreni, tipi di danno, stati, requisiti e sinergie, purché il progetto mantenga una tassonomia governata. ⁴

Componenti logici della cella. Il termine “componente” nel dominio della mappa indica un frammento dati, non necessariamente un `UActorComponent`.

Componente logico	Responsabilità
Traversal	Costo, blocchi e requisiti di attraversamento
Surface	Materiale, conducibilità, combustibilità
Height	Quota, dislivello e vantaggio elevato
Cover	Protezione direzionale e distruttibilità
Visibility	Opacità, fumo, illuminazione
Hazard	Danno, controllo o rischio
Interaction	Azioni contestuali disponibili
Trigger	Condizioni e conseguenze
Occupancy	Unità e oggetti presenti
Acoustic	Rumore e propagazione, feature post-MVP

Sistemi ambientali. I dati delle celle vengono processati da sistemi indipendenti:

Sistema	Input	Output
MovementSystem	Unità, arco, celle	Percorribilità e costo
VisibilitySystem	Celle, quota, ostacoli	Visibilità e occultamento
FireSystem	Combustibilità, fuoco, vento	Propagazione e danno
WaterSystem	Volume, pendenza, aperture	Celle bagnate o allagate
ElectricitySystem	Conducibilità, acqua	Propagazione elettrica
IceSystem	Acqua, temperatura	Superficie ghiacciata
InteractionSystem	Azione, oggetto, requisiti	Modifica della mappa
ObjectiveSystem	Celle e unità	Controllo e punteggio

L'MVP implementa soltanto movimento, quota, copertura, visibilità, fuoco, acqua ed elettricità. Gli altri sistemi entrano dopo che le interazioni fondamentali sono leggibili.

Pathfinding. Il pathfinding autorevole deve operare sul grafo tattico, non sulla sola NavMesh. A *nasce come ricerca euristica di percorsi a costo minimo su grafi*; HPA riduce il problema tramite cluster gerarchici, mentre D* Lite è progettato per riutilizzare il lavoro quando i costi degli archi cambiano. Recast/Detour costruisce invece mesh navigabili poligonali, anche suddivise in tile, ed è particolarmente utile per movimento continuo e ambienti geometrici. ⁵

Approccio	Vantaggi	Limiti per RefactorTactics	Decisione
NavMesh/ Recast	Ottimo movimento geometrico e smoothing	Non rappresenta naturalmente celle semantiche, turni e costi discreti complessi	Supporto visivo, non autorità tattica

Approccio	Vantaggi	Limiti per RefactorTactics	Decisione
A* su griglia	Semplice, deterministico, leggibile	Può espandere molti nodi su mappe grandi	Scelta MVP
A* su grafo multilivello	Modella scale, portali, salti e requisiti	Richiede authoring e debug dedicati	Scelta principale
HPA*	Riduce ricerche su mappe grandi	Introduce cache e astrazioni da invalidare	Alpha, se necessario
D* Lite	Efficiente con cambi frequenti dei costi	Più complesso; vantaggio ridotto su mappe piccole a turni	Valutazione post-alpha
Flow field	Utile per molte unità con stessa destinazione	Poco adatto a costi fortemente dipendenti dall'unità	Non prioritario

Unreal include anche `FGraphAStar` nell'AI Module, tramite `GraphAStar.h`. Il progetto può utilizzarlo come implementazione di base, mantenendo propri graph adapter e query filter. ⁶

Il costo di attraversamento è calcolato con interi o fixed-point per evitare differenze tra piattaforme:

```
TraversalCost =
    BaseEdgeCost
+ TerrainCost
+ HeightCost
+ HazardCost
+ ExposurePreference
+ UnitModifiers
+ TemporaryEffects
```

Il pathfinder deve distinguere:

Costo	Descrizione
Costo fisico	Punti movimento realmente consumati
Costo tattico	Preferenza usata da AI o suggerimenti
Rischio	Danno o controllo atteso
Validità	Requisiti rigidi che rendono l'arco disponibile o vietato

Il giocatore umano riceve per default il percorso dal costo fisico minimo. Un'opzione UI può mostrare alternative "sicura", "coperta" o "rapida", senza modificare le regole.

API proposta:

```
USTRUCT(BlueprintType)
struct FRTCellId
```

```

{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 X = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 Y = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 Layer = 0;

    bool operator==(const FRTCellId& Other) const
    {
        return X == Other.X
            && Y == Other.Y
            && Layer == Other.Layer;
    }
};

```

```

USTRUCT(BlueprintType)
struct FRTPathQuery
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FRTCellId Start;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FRTCellId Goal;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FGameplayTagContainer UnitTags;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 MovementBudget = 6;
};

```

```

UCLASS()
class REFACTORTACTICS_API URTPathfindingSubsystem
: public UWorldSubsystem
{
    GENERATED_BODY()

public:
    UFUNCTION(BlueprintCallable, Category = "RefactorTactics|Pathfinding")
    bool FindPath(
        const FRTPathQuery& Query,
        TArray<FRTCellId>& OutPath,

```



```
int32& OutCost) const;
};
```

Contratto funzionale:

```
FindPath(Query)
→ controlla la revisione del grafo
→ filtra gli archi non validi
→ interroga i cost provider registrati
→ esegue A*
→ restituisce percorso, costo e motivi di fallimento
```

La cache utilizza almeno `GraphRevision`, `UnitTraversalProfile`, `Start`, `Goal` e `MovementBudget`. Un evento dinamico incrementa la revisione dei chunk interessati, non necessariamente dell'intera mappa.

Line of sight e tiro. LOS, pathfinding e traiettoria sono servizi separati. La LOS logica deve attraversare le celle interessate e considerare quota, copertura, fumo e opacità. Una successiva trace Unreal può confermare collisioni geometriche tra punti di tiro e bersaglio. Unreal espone line trace, shape trace e simulazioni di traiettorie con collisione; queste funzioni sono utili per la validazione geometrica e per le preview visive. ⁷

```
CanTarget
├─ gittata valida
├─ cella bersaglio nota
├─ LOS logica valida
├─ regole di copertura
├─ requisiti dell'abilità
└─ trace geometrica, quando richiesta
```

La quota modifica il tiro attraverso regole esplicite, non bonus impliciti difficili da capire. Esempio:

Condizione	Effetto
Attaccante più alto	Ignora copertura bassa adiacente al bersaglio
Dislivello eccessivo	Bersaglio fuori arco per armi specifiche
Fumo tra due celle	Tiro diretto vietato o penalizzato
Parete distrutta	LOS aggiornata nello stesso turno dopo la distruzione
Tiro ad arco	Può superare ostacoli bassi ma non soffitti

Interazioni tra abilità e mappa. Ogni effetto deve dichiarare target, tag richiesti, tag applicati e durata.

Abilità	Risultato
Muro di ghiaccio	Crea ostacolo, copertura e archi bloccati

Abilità	Risultato
Granata fumogena	Aggiunge opacità e limita il targeting
Campo gravitazionale	Aumenta il costo degli archi nell'area
Portale	Registra un arco temporaneo tra due celle
Esplosivo	Danneggia unità e componenti distruttibili
Fulmine	Si propaga nelle celle con tag conduttivi
Congelamento	Trasforma acqua in terreno solido e scivoloso
Spinta	Sposta un'unità lungo archi forzati, con collisione

Gli effetti ambientali sono risolti tramite eventi, per esempio:

```
AbilityImpact
→ AddTag(Cell, State.Wet)
→ ElectricitySystem riceve CellChanged
→ cerca celle conduttive connesse
→ applica effetto alle unità presenti
→ registra il risultato nel TurnLog
```

Eventi dinamici della mappa. Gli eventi possono essere programmati, attivati da abilità o causati da obiettivi.

Evento	Conseguenza tecnica
Ponte distrutto	Rimozione di archi e aggiornamento LOS
Porta aperta	Attivazione di archi e riduzione opacità
Ascensore spostato	Cambio dei collegamenti verticali
Stanza allagata	Modifica superficie e conducibilità
Incendio	Hazard, visibilità e propagazione
Crollo	Nuove celle bloccate e possibili cadute
Terminale violato	Modifica obiettivo o sistema di sicurezza

Ogni evento deve produrre un `MapChangeSet` serializzabile:

```
MapChangeSet
├─ TurnNumber
├─ SourceId
├─ ChangedCells[]
├─ AddedEdges[]
├─ RemovedEdges[]
```

```
└─ AppliedTags[]
└─ NewGraphRevision
```

Pianificazione e intenti condivisi. Durante la pianificazione ogni giocatore vede:

Elemento	Alleato	Nemico
Percorso pianificato	Sì	No
Cella finale	Sì	No
Abilità selezionata	Sì	No
Bersaglio	Sì	No
Area d'effetto	Sì	No
Direzione di dash/spinta	Sì	No
Label dell'intento	Sì	No
Stato "pronto"	Sì	No
Cursore esatto	Opzionale	No
Esito dopo la risoluzione	Sì	Sì

Gli intenti alleati devono aggiornarsi quasi in tempo reale, con una frequenza obiettivo di 10 aggiornamenti al secondo e interpolazione grafica lato client. Non è necessario replicare ogni movimento del mouse.

Struttura proposta:

```
USTRUCT(BlueprintType)
struct FRTPlannedIntent
{
    GENERATED_BODY()

    UPROPERTY(BlueprintReadWrite)
    int32 Sequence = 0;

    UPROPERTY(BlueprintReadWrite)
    FGuid UnitId;

    UPROPERTY(BlueprintReadWrite)
    TArray<FRTCellId> PlannedPath;

    UPROPERTY(BlueprintReadWrite)
    FPrimaryAssetId AbilityId;

    UPROPERTY(BlueprintReadWrite)
    FRTCellId TargetCell;
```

```

UPROPERTY(BlueprintReadWrite)
FGameplayTag IntentLabel;

UPROPERTY(BlueprintReadWrite)
bool bReady = false;
};

```

Le label minime sono:

```

Intent.Focus
Intent.Protect
Intent.Scout
Intent.Trap
Intent.Wait
Intent.Escape
Intent.Interact
Intent.Control

```

Warning di conflitto. Il sistema segnala ma non impedisce automaticamente:

Warning	Severità
Due alleati terminano nella stessa cella	Alta
Percorsi alleati si incrociano nello stesso micro-step	Media
Area alleata può colpire un compagno	Alta
Un alleato modifica una cella necessaria al tuo percorso	Alta
Due giocatori usano la stessa risorsa limitata	Media
Un'azione interrompe una combo dichiarata	Media
Il bersaglio previsto non sarà più visibile	Informativa
Il piano dipende da un esito incerto	Informativa

I warning non devono rivelare dati nemici. Il calcolo usa esclusivamente stato pubblico e intenti della propria squadra.

Ping rapidi. Il menu contestuale include attacco, difesa, movimento, pericolo, attesa, interazione e richiesta di aiuto. I ping possiedono posizione, autore, tipo, durata e opzionale riferimento a cella, unità o obiettivo.

Disegno tattico. I giocatori possono tracciare frecce e linee temporanee. Il disegno è:

- visibile soltanto alla squadra;
- limitato in frequenza e lunghezza;
- cancellabile dall'autore;
- eliminato all'inizio della risoluzione;
- disattivabile nelle impostazioni;

- soggetto a mute e strumenti anti-abuso.

Indicatori di stato.

Stato	Significato
Pianificazione	Nessun intento confermato
Modifica	L'intento è cambiato di recente
Provvisorio	Intento presente, non confermato
Pronto	Intento confermato
Disconnesso	Ultimo intento conservato temporaneamente
Auto-ready	Piano confermato dal sistema alla scadenza

Quando tutti sono pronti il turno può iniziare dopo un countdown breve, per esempio 1,5 secondi, durante il quale è ancora possibile annullare il ready.

Sistema dei personaggi. Ogni personaggio è composto da:

```

Personaggio
├─ Identità base
├─ Statistiche base
├─ Kit fondamentale
├─ Moduli abilità
├─ Talenti
├─ Equipaggiamento tattico
├─ Specializzazione
├─ Trattati
├─ Affinità ambientali
└─ Ultimate

```

Il kit fondamentale garantisce riconoscibilità. Un personaggio non può cambiare simultaneamente arma primaria, mobilità, difesa e ultimate fino a diventare indistinguibile dagli altri.

Slot	Quantità equipaggiata	Pool target
Variante attacco	1	2-3
Variante mobilità	1	2
Variante difesa/utility	1	2-3
Ultimate	1	2
Talenti	3	9-12
Gadget	1	4-6 compatibili
Specializzazione	1	2-3

Slot	Quantità equipaggiata	Pool target
Tratto minore	1	3-5

Esempio di variazioni della stessa abilità:

Granata base	Variante
Impatto	Esplode immediatamente
Ritardata	Esplode nel turno successivo
Fumogena	Rimuove il danno e crea fumo
Mine	Distribuisce tre cariche più piccole
Conduttiva	Bagna l'area e prepara sinergie elettriche

Le varianti devono scambiare vantaggi e svantaggi. Non sono accettabili upgrade che conservano tutti i benefici della versione base aggiungendo soltanto potenza.

Specializzazioni. Una specializzazione modifica il ruolo senza cambiare completamente l'identità. Per esempio:

Personaggio da ricognizione	Focus
Recon	Visione, sensori e mobilità
Ambusher	Trappole e attacchi da posizione
Guardian	Copertura e protezione degli alleati

Equipaggiamento tattico. I gadget sono strumenti situazionali: drone, sensore, mina, beacon, kit medico, barriera, rampino o scanner. Non devono essere semplici bonus passivi alle statistiche.

Affinità con la mappa. Le affinità modificano le interazioni ambientali:

Affinità	Effetto
Acquatico	Ignora parte del costo dell'acqua
Piromante	Resiste al fuoco e lo utilizza come risorsa
Tecnico	Interagisce più rapidamente con dispositivi
Montanaro	Riduce il costo su roccia e dislivelli
Conduttivo	Potenzia elettricità ma aumenta rischi specifici
Ombra	Ottiene vantaggi in celle poco illuminate

Le affinità non devono rendere obbligatorio un personaggio su una determinata mappa. Ogni ostacolo strategico deve avere più soluzioni.

Perk di squadra. Prima della partita il team sceglie un perk condiviso, dopo la selezione dei personaggi ma prima della schermata finale di caricamento.

Perk	Funzione
Rete di sensori	Migliora la ricognizione iniziale
Kit logistico	Una carica gadget aggiuntiva condivisa
Fortificazione	Copertura iniziale su una zona definita
Riserva energetica	Recupero risorsa al primo turno
Protocollo medico	Rientro o cura leggermente migliorati

I perk sono visibili alla squadra avversaria al caricamento, così da evitare vantaggi nascosti non leggibili.

Progressione. La progressione permanente sblocca possibilità, non potenza numerica.

Consentito	Non consentito nel competitivo
Nuovi moduli alternativi	Danno permanente superiore
Nuovi gadget equivalenti	Più salute per anzianità account
Specializzazioni	Riduzione permanente dei cooldown
Skin, animazioni, banner	Precisione superiore sbloccabile
Linee vocali ed emote	Slot extra esclusivi
Modalità e mutatori	Perk numericamente migliori

Vincoli di bilanciamento.

- Ogni build deve conservare almeno una debolezza rilevante.
- Nessun personaggio deve ignorare contemporaneamente movimento, LOS e pericoli ambientali.
- Le combo ambientali più forti richiedono almeno due azioni, una condizione della mappa o un rischio evidente.
- Il danno massimo prevedibile deve essere mostrabile dalla UI.
- Gli effetti di controllo devono avere contromisure o costi significativi.
- Le varianti non devono moltiplicare in modo incontrollato il numero di interazioni da testare.
- Il competitivo usa un catalogo di contenuti approvato e versionato.

[Scarica il diagramma vettoriale del ciclo di turno](#)

Architettura tecnica e networking

Stack raccomandato.

Livello	Tecnologia
Motore	Unreal Engine 5, release stabile

Livello	Tecnologia
Core simulazione	C++
Prototipi e presentazione	Blueprints
Abilità e status	Gameplay Ability System
Tassonomia	Gameplay Tags
Input	Enhanced Input
UI	UMG; CommonUI dopo una prova tecnica
Dati	Primary Data Assets, Data Registry, JSON per import/mod
Multiplayer	Replication Unreal, RPC e server autorevole
Servizi online	Interfaccia astratta; EOS o piattaforma in una fase successiva
Build	Unreal Build Tool e Automation Tool
Version control	Git LFS per team piccolo; valutare Perforce con crescita del team
Test	Automation Tests, functional tests e simulatori headless

Enhanced Input supporta azioni astratte, mapping context prioritari, modificatori, trigger e rimappatura; è quindi adatto a distinguere input di mappa, targeting, disegno e UI. ⁸

UMG utilizza Widget Blueprint per layout e logica UI. CommonUI offre strumenti per interfacce multilivello, routing degli input e supporto multiplatforma, ma deve essere adottato soltanto dopo un piccolo proof of concept, evitando di introdurlo in tutte le schermate prima di averne validato il workflow. ⁹

C++ contro Blueprint.

Area	C++	Blueprint	Scelta
Simulatore del turno	Controllo, testabilità, determinismo	Difficile da revisionare su grafi grandi	C++
Pathfinding	Migliore per algoritmi e profiling	Utile per debug e parametri	C++ con API Blueprint
Sistemi ambientali	Core in codice	Configurazione ed effetti visivi	Ibrido
Abilità	Base e validazione	Composizione e presentazione	GAS ibrido
UI	View model e servizi	Layout e animazioni	Ibrido
Personaggi	Contratti e componenti base	Asset e varianti	Ibrido
Mod content	Loader e validator	Asset data-only	Core C++, contenuti dati

Area	C++	Blueprint	Scelta
Networking	Replication e sicurezza	Prototipi limitati	C++
VFX e audio	Solo API essenziali	Iterazione rapida	Blueprint/Niagara
Regole classificate	Più controllabili	Rischio di logica dispersa	C++ o dati validati

Epic evidenzia che C++ fornisce maggiore controllo su prestazioni, rete, serializzazione e API, mentre Blueprint facilita creazione e iterazione. RefactorTactics adotterà quindi la regola: **C++ definisce ciò che è possibile; dati e Blueprint definiscono quale variante viene utilizzata.** ¹⁰

C# nel progetto. Unreal non usa C# come linguaggio gameplay principale. L'esperienza C# resta utile per classi, interfacce, generics, eventi e architettura, ma occorre imparare reflection Unreal, macro, garbage collection, moduli e lifecycle degli UObject. La guida ufficiale per sviluppatori provenienti da Unity è un riferimento utile per la traduzione concettuale. ¹¹

C#	Unreal C++
<code>class</code> serializzabile	<code>UCLASS()</code>
<code>struct</code> dati	<code>USTRUCT()</code>
attributo/proprietà editor	<code>UPROPERTY()</code>
metodo esposto	<code>UFUNCTION()</code>
<code>List<T></code>	<code>TArray<T></code>
<code>Dictionary<K,V></code>	<code>TMap<K,V></code>
evento/delegate	Delegate Unreal
riferimento managed	<code>TObjectPtr<T></code> o soft reference
interface	<code>UINTERFACE</code> + interfaccia C++
assembly	Module Unreal
reflection runtime	Unreal Header Tool

Struttura dei moduli C++.

```
Source/
├── RefactorTactics/
│   ├── Core/
│   ├── Characters/
│   ├── Map/
│   ├── Planning/
│   ├── Simulation/
│   └── UI/
├── RefactorTacticsEditor/
└── Validators/
```

```

|   |— MapTools/
|   |— ModTools/
|— RefactorTacticsTests/

```

Per il primo anno può essere mantenuto un singolo modulo runtime con cartelle interne. La suddivisione in più moduli avviene quando le dipendenze e i tempi di compilazione lo giustificano.

Plugin Unreal raccomandati.

Plugin	Utilizzo	Fase
GameplayAbilities	Abilità, costi, cooldown, effetti	Vertical slice
GameplayTags	Tassonomia e query	Da subito
GameplayTasks	Task asincroni di abilità	Vertical slice
EnhancedInput	Input e context	Da subito
TargetingSystem	Valutare preset di targeting	Alpha
CommonUI	UI e routing cross-input	Dopo proof of concept
DataRegistry	Cataloghi di dati	Alpha
GameFeatures	Pacchetti modulari e mod	Beta/mod release
OnlineSubsystemEOS o Online Services	Sessioni e servizi	Alpha
Niagara	VFX ambientali e abilità	Vertical slice

Il Gameplay Ability System richiede l'attivazione del plugin e le dipendenze `GameplayAbilities`, `GameplayTags` e `GameplayTasks`. Il sistema supporta ability, task, attributi ed effetti, inclusi costi e cooldown. ¹²

Esempio `Build.cs`:

```

PublicDependencyModuleNames.AddRange(
    new string[]
    {
        "Core",
        "CoreUObject",
        "Engine",
        "InputCore",
        "EnhancedInput",
        "UMG",
        "AIModule",
        "GameplayAbilities",
        "GameplayTags",
        "GameplayTasks"
    });

```

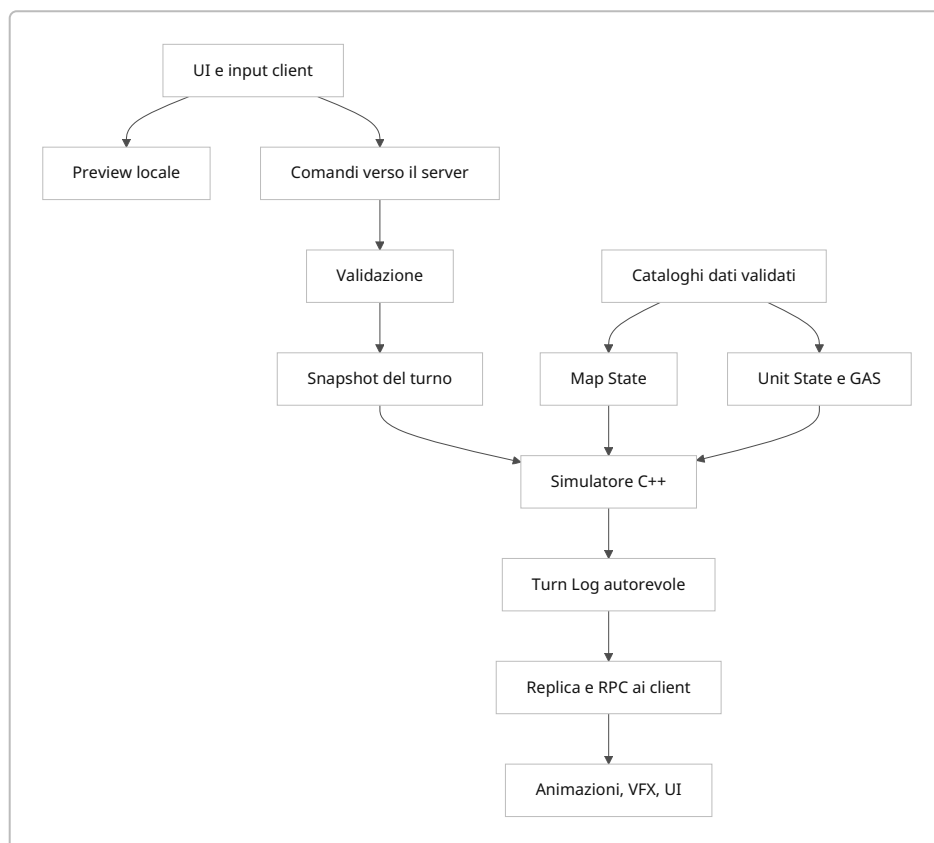
Questo file è scritto in C# perché Unreal Build Tool utilizza script di build C#, ma il gameplay continua a essere implementato in Unreal C++.

Simulatore autorevole. Lo stato logico deve essere separato dalla rappresentazione visuale.

Authoritative Match State

- └ MapState
- └ Units
- └ Effects
- └ Objectives
- └ TurnState
- └ CommandSnapshot
- └ RandomSeed
- └ TurnLog

Le animazioni non decidono quando un colpo avviene. Il simulatore produce eventi temporizzati; il client riproduce animazioni e VFX corrispondenti.



[Scarica il diagramma vettoriale dell'architettura](#)

Modello di rete.

Modello	Vantaggi	Svantaggi	Decisione
Peer-to-peer con host	Costi infrastrutturali iniziali ridotti	Host advantage, migrazione e sicurezza complesse	Solo prototipo locale
Listen server	Semplice per test e partite private	Un giocatore è autorità e host	Supporto custom
Dedicated authoritative	Equità, anti-cheat e stato centrale	Costo operativo e deployment	Produzione competitiva
Lockstep peer	Poco traffico per simulazioni deterministiche	Ritardo del peer peggiore e gestione disconnessioni	Non principale
Server simulation con snapshot	Controllo completo e replay	Maggiore lavoro backend	Scelta

In Unreal gli Actor possono replicare proprietà e chiamate di funzione; la property replication e gli RPC sono i meccanismi fondamentali per distribuire stato dal server ai client. ¹³

Separazione della visibilità. Non è sufficiente nascondere graficamente gli intenti avversari: tali intenti non devono essere replicati al loro processo.

Architettura proposta:

```

Server
├── CanonicalIntentStore
│   ├── Team A intents
│   └── Team B intents
├── TeamIntentRelay A
│   └── snapshot sanitizzati inviati soltanto ai client A
└── TeamIntentRelay B
    └── snapshot sanitizzati inviati soltanto ai client B

```

Una semplice proprietà replicata su un Actor globale è vietata per gli intenti. Il server invia snapshot di squadra tramite:

- Client RPC indirizzati ai singoli compagni; oppure
- subobject/Actor relay specifici per ogni client, contenenti la sola vista autorizzata.

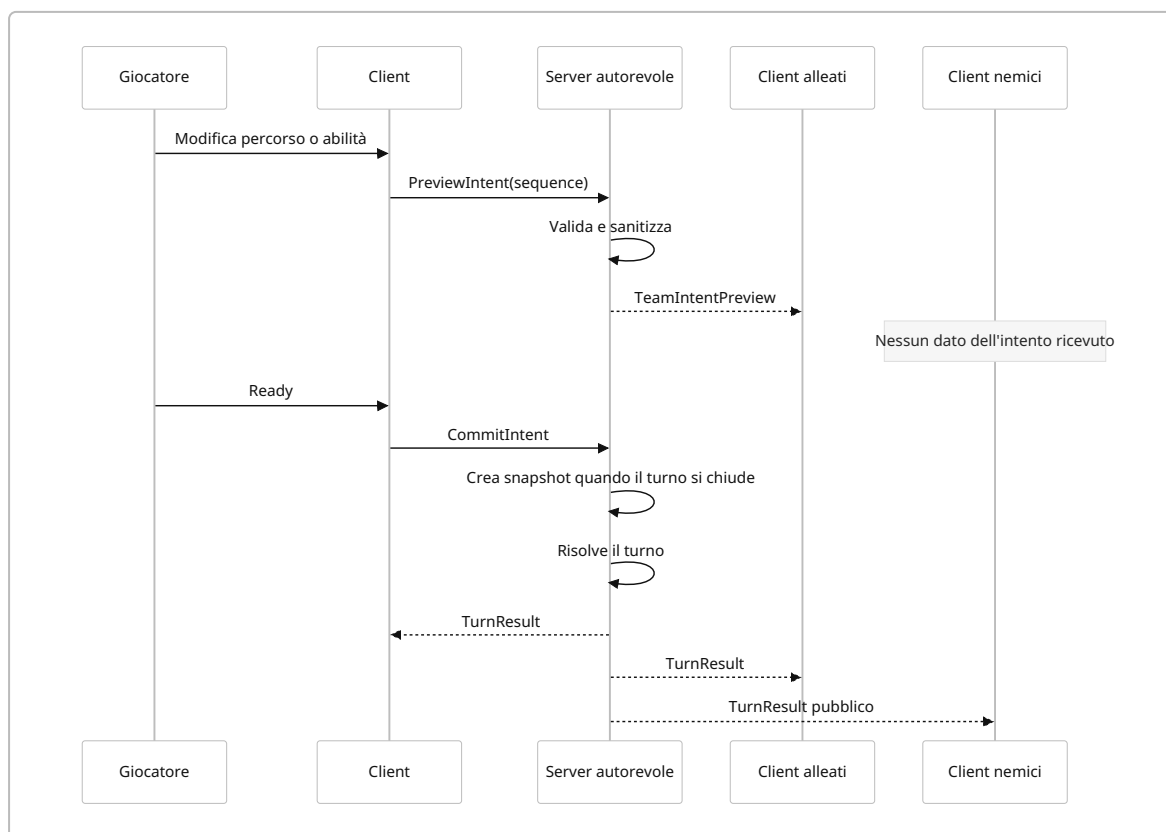
Il server conserva sempre il dato canonico completo.

Affidabilità dei messaggi.

Messaggio	Direzione	Affidabilità
Preview percorso	Client → server	Unreliable, sequenziato
Preview abilità	Client → server	Unreliable, sequenziato

Messaggio	Direzione	Affidabilità
Broadcast preview al team	Server → client	Unreliable, sequenziato
Ping	Client → server → team	Reliable con rate limit
Ready/unready	Client → server	Reliable
Commit finale	Client → server	Reliable
Snapshot del turno	Server interno	Immutabile
Risultato autorevole	Server → client	Reliable o stato replicato
Disegno tattico	Unreliable, con chunk e scadenza	Unreliable

I pacchetti preview includono un `SequenceNumber`; i client ignorano aggiornamenti più vecchi.

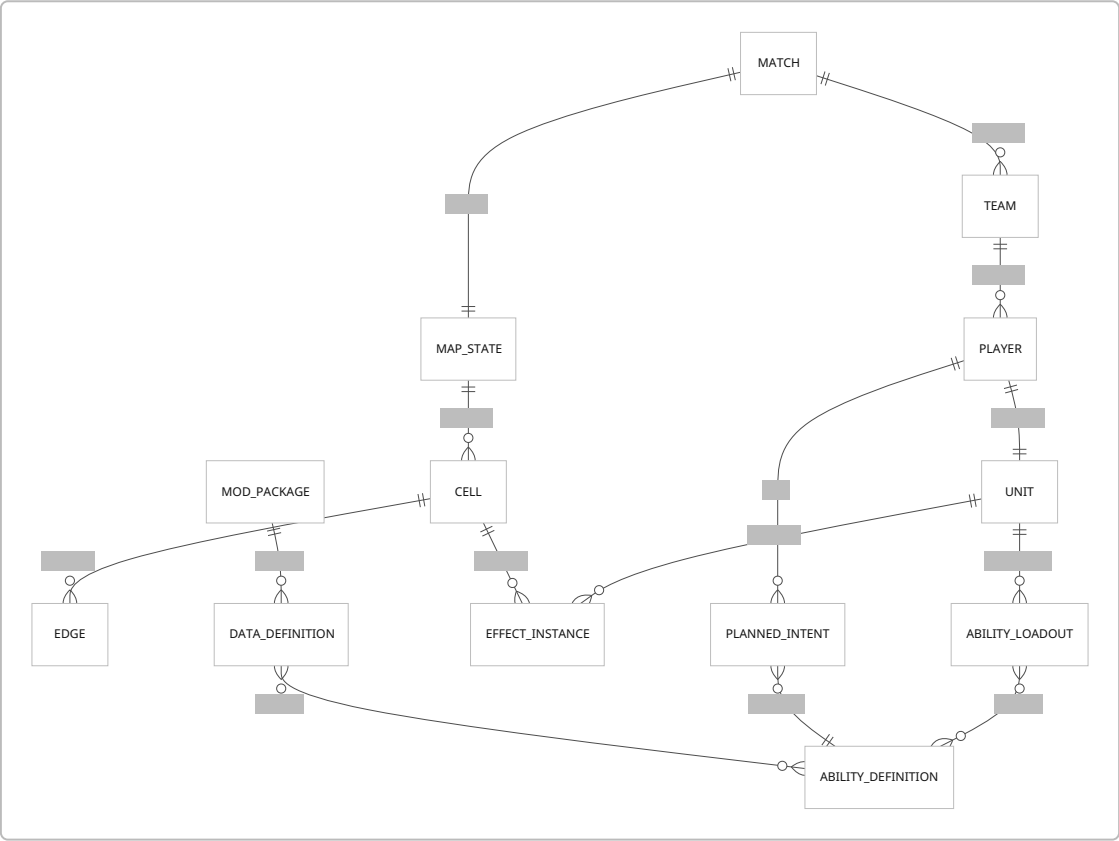


Determinismo e replay. Il progetto deve utilizzare:

- ordine stabile degli ID;
- costi interi;
- seed registrato per ogni turno;
- niente iterazione dipendente dall'ordine non garantito di mappe hash;
- niente risultato gameplay deciso da frame rate o animazione;
- versioni esplicite di regole e contenuti;
- command log per ricostruire una partita.

Un replay registra `MatchConfig`, hash dei contenuti, seed iniziale, snapshot degli intenti e risultati per turno. Durante le prime milestone è sufficiente un replay logico senza registrazione video.

Entity relationship.



[Scarica il diagramma vettoriale delle entità](#)

Requisiti non funzionali iniziali.

Requisito	Target
Frame rate client	60 FPS su hardware target medio
Tick simulazione	Non legato al frame rate
Path query singola	Mediana inferiore a 2 ms su mappa MVP
Preview completa	Inferiore a 50 ms lato client
Update intenti	8-12 Hz
Riconnessione	Entro due turni, target alpha
Turn resolution	Inferiore a 100 ms server per match MVP
Crash-free sessions	Oltre 99% in beta
Divergenze replay	Zero nei test deterministici
Intent leak test	Zero pacchetti non autorizzati

Modding e contratti dati

Il supporto alle mod viene pubblicato dopo MVP, alpha e beta. Tuttavia tre decisioni devono essere applicate dal primo giorno:

1. ogni contenuto possiede un ID stabile;
2. le regole configurabili risiedono in asset o strutture dati;
3. il caricamento passa da cataloghi e validator, non da riferimenti hard-coded.

Unreal consente di gestire contenuti attraverso Asset Manager, Primary Data Asset, Primary Asset Label e Data Registry. I Gameplay Tags forniscono classificazione gerarchica, mentre il Game Features subsystem gestisce plugin di feature installabili e disinstallabili. Questi sistemi costituiscono una base utile, ma RefactorTactics necessita comunque di manifest, schema, validazione, dipendenze e policy proprie. ¹⁴

Livelli di modding.

Livello	Contenuto	Supporto
Data mod	Terreni, valori, tag, tabelle	Prima release mod
Asset mod	Mesh, materiali, audio, VFX	Prima release mod, con limiti
Blueprint sandbox	Abilità basate su nodi approvati	Release successiva
Map mod	Celle, layer, oggetti e obiettivi	Prima o seconda release mod
Mode mod	Regole e scoring data-driven	Seconda release mod
Native plugin	C++ arbitrario	Non supportato nel client standard

Il codice nativo di terze parti non viene caricato nel client retail per ragioni di sicurezza, stabilità e compatibilità. Server e client devono avere lo stesso catalogo di mod, verificato tramite hash.

Manifest.

```
{
  "schemaVersion": 1,
  "modId": "aaron.refactortactics.volcanic_pack",
  "displayName": "Volcanic Pack",
  "version": "1.0.0",
  "gameVersion": "0.8.x",
  "type": "Content",
  "dependencies": [
    {
      "modId": "refactortactics.core",
      "version": ">=0.8.0"
    }
  ],
  "content": [
    "Terrains/Lava.json",
    "Abilities/MagmaWall.json"
  ]
}
```

```

    ],
    "networkPolicy": "ServerRequired"
}

```

Definizione di terreno.

```

{
  "schemaVersion": 1,
  "id": "Terrain.Lava",
  "displayName": "Lava",
  "tags": [
    "Terrain.Liquid",
    "Hazard.Fire",
    "Surface.Hot"
  ],
  "movement": {
    "baseCost": 5,
    "blockedByDefault": false,
    "requiredTags": [],
    "forbiddenTags": [
      "Unit.Trait.Flammable"
    ]
  },
  "visibility": {
    "blocksLineOfSight": false,
    "opacity": 0.1
  },
  "turnEffects": [
    {
      "effectId": "Effect.Burning",
      "magnitude": 20
    }
  ]
}

```

Definizione di abilità.

```

{
  "schemaVersion": 1,
  "id": "Ability.MagmaWall",
  "abilityFamily": "Control.Wall",
  "range": 5,
  "cost": {
    "energy": 2,
    "cooldownTurns": 3
  },
  "targeting": {
    "type": "LineOfCells",
    "length": 3,
  }
}

```



```

    "requiresLineOfSight": true
  },
  "effects": [
    {
      "type": "AddCellTag",
      "tag": "Hazard.Fire",
      "durationTurns": 2
    },
    {
      "type": "BlockEdges",
      "durationTurns": 2
    }
  ]
}

```

API concettuale.

```

IModContentProvider
├─ GetManifest()
├─ GetDefinitions()
├─ GetDependencies()
└─ GetContentHash()

IDataDefinitionValidator
├─ ValidateSchema()
├─ ValidateReferences()
├─ ValidateGameplayTags()
├─ ValidateNetworkPolicy()
└─ ValidateBalanceLimits()

ITraversalCostProvider
├─ CanTraverse(Context, Edge)
└─ GetAdditionalCost(Context, Edge)

IMapEffectProcessor
├─ CanProcess(Event)
└─ Process(Event, ChangeSet)

```

Le interfacce di runtime sono implementate dal gioco. Una mod data-only seleziona provider già approvati e passa loro configurazioni. Non può inserire funzioni arbitrarie.

Versionamento.

Cambiamento	Strategia
Aggiunta campo opzionale	Minor schema
Ridenominazione campo	Redirect e migration

Cambiamento	Strategia
Cambio semantico	Major schema
Rimozione di un ID	Deprecation per almeno una release
Ridenominazione tag	Gameplay Tag redirect
Mod non compatibile	Disabilitazione con errore leggibile
Save vecchio	Pipeline di migration
Match online	Hash esatto dei contenuti

Validator. Il toolkit mod deve verificare:

- ID duplicati;
- dipendenze mancanti o circolari;
- tag sconosciuti;
- riferimenti ad asset inesistenti;
- costi negativi;
- loop infiniti dichiarativi;
- aree e durate oltre limiti;
- path impossibili;
- mappe senza spawn o obiettivi validi;
- asset oltre budget;
- contenuti non consentiti nelle playlist classificate.

Distribuzione. La prima versione può utilizzare una cartella locale e pacchetti importabili. L'integrazione con Steam Workshop o altri cataloghi viene trattata come adapter di distribuzione, non come fondamento del formato.

```
RefactorTactics/
├─ Mods/
│   └─ VolcanicPack/
│       ├── manifest.json
│       ├── Content/
│       ├── Data/
│       └─ Localization/
```

Networking delle mod. All'ingresso in una sessione:

```
Client invia ContentCatalogHash
→ server confronta hash
→ se uguale: accesso
→ se mancano mod: elenco dipendenze
→ se versione incompatibile: rifiuto motivato
→ se catalogo classificato: ignora contenuti non approvati
```

Le partite classificate accettano soltanto contenuti firmati e inclusi nel catalogo ufficiale della stagione. Le partite custom possono usare mod locali, ma devono mostrare chiaramente che progressione, ranking e statistiche competitive sono disattivati.

Percorso tutorial e configurazione

Il percorso formativo usa il gioco come progetto didattico. Ogni lezione deve produrre una parte eseguibile, introdurre un numero limitato di concetti e terminare con una verifica concreta.

Prerequisiti.

Requisito	Minimo
Sistema operativo consigliato	Windows per il primo percorso
Unreal Engine	Ultima release stabile scelta dal progetto
IDE	Visual Studio 2022
Workload VS	Game development with C++
SDK	Windows SDK compatibile con la versione UE
Version control	Git e Git LFS
Spazio libero	Almeno 100 GB consigliati per motore, cache e progetto
Conoscenze	C# base, classi, interfacce, collection, eventi

Visual Studio 2022 è il toolchain predefinito per le release moderne di Unreal; la configurazione ufficiale richiede il workload C++ per game development e un Windows SDK compatibile. ¹⁵

Configurazione passo per passo.

Passo	Operazione	Risultato
Installazione	Installare Epic Games Launcher	Accesso alle versioni UE
Motore	Installare una versione stabile di UE5	Editor disponibile
IDE	Installare Visual Studio 2022	Compiler C++ disponibile
Workload	Selezionare Game development with C++	Toolchain completa
Progetto	Games → Blank → C++ → Desktop	Progetto compilabile
Nome	<code>RefactorTactics</code>	Namespace coerente
Repository	Inizializzare Git	Cronologia attiva
LFS	Tracciare <code>.uasset</code> , <code>.umap</code> , audio e texture	Binary asset gestiti
Primo commit	Compilare e aprire l'editor	Baseline riproducibile
Plugin	Attivare Gameplay Tags ed Enhanced Input	Fondazioni gameplay
Test	Creare una mappa <code>L_DevSandbox</code>	Area di esercizio

Non conviene iniziare compilando Unreal dal sorgente. La build binaria tramite Launcher è l'approccio più semplice; l'accesso al sorgente diventa utile soltanto quando è necessario ispezionare o modificare il motore. ¹⁶

Struttura dei contenuti.

```
Content/RefactorTactics/
├─ Characters/
├─ Core/
├─ Data/
├─ Input/
├─ Maps/
├─ Materials/
├─ UI/
├─ VFX/
└─ Tests/
```

Le cartelle non devono essere organizzate per nome del singolo sviluppatore. Ogni asset usa un prefisso coerente, per esempio `BP_`, `WBP_`, `DA_`, `GA_`, `GE_`, `IA_`, `IMC_`, `M_` e `T_`.

Roadmap delle lezioni.

Lezione	Prodotto costruito	Concetti
Ambiente e primo progetto	Progetto compilabile e repository	Editor, IDE, build, commit
Unreal per chi conosce C#	Actor C++ esposto a Blueprint	UCLASS, UPROPERTY, lifecycle
Camera tattica	Pan, zoom e rotazione	Pawn, Controller, Enhanced Input
Prima cella	Cella selezionabile	Actor, collisione, mouse trace
Generatore di griglia	Mappa 2D graybox	Loop, spawn, coordinate
Identità delle celle	<code>FRTCellId</code> e lookup	USTRUCT, TArray, TMap
Grafo di movimento	Vicini e archi	Strutture dati e debug
Primo A*	Percorso tra due celle	Heuristic, open/closed set
Budget di movimento	Evidenziazione celle raggiungibili	Query e costi
Mappa multilivello	Scale e ponte	Layer e transizioni
Dati del terreno	Acqua, roccia, fuoco	Data Asset e Gameplay Tags
Personaggio tattico	Selezione e movimento	Pawn, stato logico, animazione
Ciclo di turno	Planning e resolution	State machine
Prima abilità	Attacco su una cella	GAS, targeting, cooldown

Lezione	Prodotto costruito	Concetti
LOS e copertura	Tiro bloccato o protetto	Trace, quota, cover
Ambiente reattivo	Acqua più elettricità	Eventi e sistemi
UI di pianificazione	Percorso e area fantasma	UMG e view model
Intenzioni alleate locali	Due finestre PIE	Separazione player/team
Multiplayer autorevole	Commit e risoluzione server	RPC, replica, authority
Warning di conflitto	Collisioni tra piani alleati	Validator di intenti
Ping e label	Comunicazione contestuale	UI e messaggi di squadra
Disegno tattico	Linee temporanee	Spline, rate limit
Personaggi modulari	Loadout e talenti	Primary Data Asset
Replay del turno	Command log riproducibile	Serializzazione e seed
Vertical slice	Partita 2 contro 2	Integrazione e playtest
Fondazioni mod	Import JSON validato	Schema e registri
Mod toolkit	Terreno o abilità custom	Packaging e validator

Formato di ogni lezione.

Obiettivo

- Concetti nuovi
- Passi nell'editor
- Codice minimo
- Test manuale
- Errore comune
- Piccola sfida
- Commit suggerito

Esempio di lezione: prima cella.

Obiettivo: creare una cella selezionabile da Blueprint.

File `RTGridCell.h`:

```
#pragma once

#include "CoreMinimal.h"
#include "GameFramework/Actor.h"
#include "RTGridCell.generated.h"

UCLASS()
class REFACTORTACTICS_API ARTGridCell : public AActor
```

```

{
    GENERATED_BODY()

public:
    ARTGridCell();

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Grid")
    int32 GridX = 0;

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Grid")
    int32 GridY = 0;

    UFUNCTION(BlueprintCallable, Category = "Grid")
    FString GetCellLabel() const;
};

```

File `RTGridCell.cpp`:

```

#include "RTGridCell.h"

ARTGridCell::ARTGridCell()
{
    PrimaryActorTick.bCanEverTick = false;
}

FString ARTGridCell::GetCellLabel() const
{
    return FString::Printf(TEXT("Cell %d, %d"), GridX, GridY);
}

```

Passi:

1. Compilare il progetto.
2. Nell'editor creare un Blueprint derivato da `RTGridCell`.
3. Chiamarlo `BP_GridCell`.
4. Aggiungere uno Static Mesh Component con un cubo piatto.
5. Impostare `GridX` e `GridY`.
6. Chiamare `GetCellLabel` da `BeginPlay`.
7. Stampare il risultato.
8. Verificare che il log mostri coordinate corrette.
9. Eseguire il commit `lesson/grid-cell`.

Esempio di lezione: costo di attraversamento.

```

int32 GetTerrainCost(const FGameplayTagContainer& CellTags)
{
    if (CellTags.HasTagExact(
        FGameplayTag::RequestGameplayTag(TEXT("Terrain.Water"))))
    {

```

```

        return 2;
    }

    if (CellTags.HasTagExact(
        FGameplayTag::RequestGameplayTag(TEXT("Hazard.Fire"))))
    {
        return 5;
    }

    return 1;
}

```

La versione didattica può usare condizioni dirette. Una lezione successiva sostituisce gli `if` con Data Asset e provider registrati.

Esempio di lezione: stato del turno.

```

UENUM(BlueprintType)
enum class ERTTurnPhase : uint8
{
    Planning,
    Committing,
    Resolving,
    Cleanup
};

```

```

UCLASS()
class REFACTORTACTICS_API ARTTurnManager
    : public AGameStateBase
{
    GENERATED_BODY()

public:
    UPROPERTY(ReplicatedUsing = OnRep_TurnPhase)
    ERTTurnPhase TurnPhase = ERTTurnPhase::Planning;

    UFUNCTION()
    void OnRep_TurnPhase();
};

```

La prima versione cambia fase con un timer. Le lezioni successive aggiungono ready, commit, snapshot e server authority.

Risorse ufficiali da seguire.

Tema	Risorsa
Passaggio da Unity/C#	Guida Unreal per sviluppatori Unity 17

Tema	Risorsa
Blueprint	Blueprint Overview ¹⁸
Bilanciamento C++ e Blueprint	Guida Epic dedicata ¹⁹
Gameplay Ability System	Documentazione GAS ¹²
Gameplay Tags	Documentazione dei tag ²⁰
Enhanced Input	Documentazione Input ²¹
UMG	UMG UI Designer Quick Start ²²
Navigazione grafi	API <code>FGraphASTar</code> ²³
A* originale	Hart, Nilsson e Raphael ²⁴
HPA*	Botea, Müller e Schaeffer ²⁵
D* Lite	Koenig e Likhachev ²⁶
Recast/Detour	Repository ufficiale ²⁷
EOS	Plugin Online Subsystem EOS ²⁸

Regole didattiche.

- Una lezione non introduce più di due macro Unreal nuove.
- Il codice da copiare deve compilare prima delle estensioni.
- Ogni snippet specifica file e classe.
- Ogni lezione termina con un test visibile.
- Il multiplayer viene introdotto dopo che il ciclo locale funziona.
- GAS viene introdotto con una sola abilità semplice.
- Le ottimizzazioni arrivano dopo profiling e test.
- Ogni refactor conserva un test o una scena dimostrativa.

Piano di sviluppo, metriche e governance

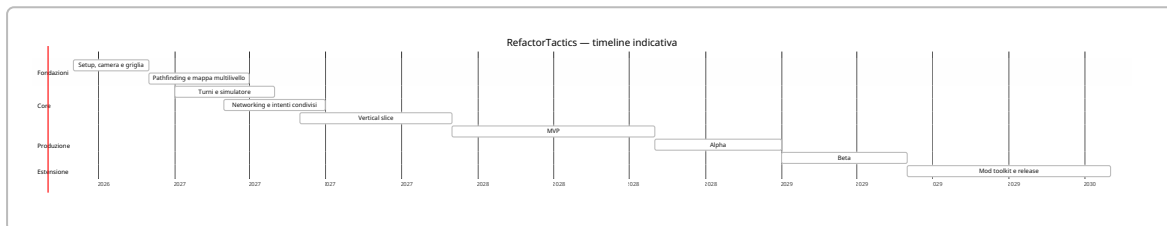
Le stime seguenti assumono un piccolo team professionale, un progetto originale senza asset AAA e sviluppo PC-first. Non includono tempi di raccolta fondi, certificazione console o campagne marketing estese.

Milestone e deliverable.

Milestone	Durata indicativa	Deliverable
Fondazioni	2-3 mesi	Camera, griglia, celle, input, repo, build automatica
Prototipo tattico	3-4 mesi	A*, movimento, turni locali, attacco, LOS
Prototipo di rete	2-3 mesi	Server authority, intenti di squadra, ready e snapshot
Vertical slice	4-6 mesi	2v2, una mappa, quattro personaggi, UI leggibile
MVP	6-9 mesi	3v3, due mappe, sei personaggi, matchmaking base

Milestone	Durata indicativa	Deliverable
Alpha	4-6 mesi	Otto personaggi, tre mappe, progressione e telemetria
Beta	4-6 mesi	Bilanciamento, onboarding, stabilità, contenuti
Mod release	6-9 mesi	Toolkit, map mod, data mod, browser/import e validator

Una produzione completa con modding può quindi richiedere circa 31-46 mesi con un team ridotto. La durata diminuisce con più personale esperto, ma cresce rapidamente se il progetto richiede qualità artistica elevata, cross-platform o live service.



Le date del diagramma sono indicative e assumono avvio operativo a settembre 2026.

Scope del vertical slice.

Area	Incluso
Formato	2 contro 2
Mappa	Una mappa graybox rifinita parzialmente
Livelli	Piano terreno, ponte e tunnel
Personaggi	Quattro
Abilità	Quattro per personaggio
Moduli	Una variante per due abilità
Ambiente	Fuoco, acqua, elettricità
UI	Path, AoE, intenti alleati, warning, ready
Networking	Dedicated server di test
Modalità	Obiettivo singolo dinamico
Audio/VFX	Minimo necessario alla leggibilità
Progressione	Nessuna
Modding	Soltanto architettura dati interna

Criteri di uscita dal vertical slice.

Criterio	Soglia
Giocatori che comprendono il ciclo dopo tutorial	Almeno 80%

Criterio	Soglia
Giocatori che notano gli intenti alleati	Almeno 90%
Turni con conflitto accidentale tra alleati	Meno del 15% dopo tre partite
Azioni con esito percepito come inspiegabile	Meno del 5%
Partite completate senza blocchi	Almeno 95%
Tempo medio decisione	Tra 15 e 30 secondi
Divergenze client/server	Zero
Leak di intenti avversari	Zero
Path query oltre budget	Meno dell'1%
Interesse a una seconda sessione	Almeno 60% dei tester target

Scope MVP.

Area	Target MVP
Formato	3 contro 3
Mappe	Due
Personaggi	Sei
Specializzazioni	Due per personaggio
Talenti	Sei o più per personaggio
Gadget	Sei condivisi
Modalità	Convergenza
Multiplayer	Sessioni private e unranked
Tutorial	Tutorial interattivo e partita contro bot
Replay	Replay logico
Progressione	Sblocchi orizzontali limitati
Modding pubblico	No

Scope alpha. L'alpha aggiunge terza mappa, otto personaggi, matchmaking più robusto, riconnessione, profili, progressione completa, telemetria, bot migliorati, strumenti di bilanciamento e primi test del pipeline mod interno.

Scope beta. La beta blocca il formato principale, riduce i cambiamenti architetturali, espande onboarding, accessibilità, localizzazione, performance, sicurezza, reportistica, moderazione e stabilità dei servizi.

Release mod. La release mod introduce:

- manifest versionato;
- import di data mod;
- editor o template per mappe;
- validatore;
- packaging;
- gestione dipendenze;
- hash di rete;
- partita custom con mod;
- documentazione API;
- esempi ufficiali;
- policy di contenuto;
- migrazione tra versioni.

Ruoli minimi.

Ruolo	Fondazioni	Produzione
Technical/game director	1	1
Gameplay programmer	1	2-3
Network/backend programmer	0,5	1-2
Technical designer	1	1-2
UI/UX designer	0,5	1
Level designer	0,5	1-2
Environment artist	0,5	1-2
Character artist/animator	Outsource o 0,5	1-2
VFX/audio	Outsource	1 combinato o outsource
QA	Condiviso	2-4
Producer	0,5	1
Community/mod tools	0	1 nella fase mod

Un team iniziale realistico è di quattro-sei persone equivalenti full-time. Durante produzione e beta può crescere a dieci-quindici, includendo QA e outsourcing.

Stima di effort per sistema.

Sistema	Effort relativo	Rischio
Camera e selezione	Basso	Basso
Griglia multilivello	Medio	Medio
Pathfinding semantico	Alto	Alto

Sistema	Effort relativo	Rischio
Simulatore dei turni	Alto	Alto
Intenti condivisi	Medio	Alto per privacy di rete
UI di previsione	Alto	Alto
Personaggi modulari	Alto	Alto per bilanciamento
GAS integration	Medio	Medio
Sistemi ambientali	Alto	Molto alto per combinazioni
Dedicated server	Alto	Alto
Matchmaking	Medio	Medio
Replay deterministico	Medio	Alto
Modding	Molto alto	Molto alto
Contenuti artistici	Molto alto	Dipende dalla qualità target

Metriche di prodotto.

Categoria	Metrica
Onboarding	Completamento tutorial
Comprensione	Percentuale di azioni correttamente previste
Coordinazione	Utilizzo di intenti, ping e label
Frizione	Numero di conflitti accidentali
Profondità	Diversità delle build utilizzate
Bilanciamento	Win rate per personaggio, mappa e build
Qualità mappe	Distribuzione delle aree utilizzate
Durata	Tempo partita e turni medi
Retention	Ritorno dopo prima e seconda sessione
Stabilità	Crash, disconnessioni, divergenze
Modding	Mod validate, installate e partite custom

Metriche di bilanciamento. Il win rate globale non è sufficiente. I dati devono essere segmentati per livello di abilità, mappa, composizione, specializzazione, perk di squadra e ordine di scelta. Una build con win rate normale ma pick rate quasi totale può essere comunque problematica.

Rischi principali.

Rischio	Impatto	Mitigazione
Troppa complessità della mappa	Il giocatore non comprende l'esito	Introdurre pochi sistemi per mappa e usare preview esplicite
Esplosione combinatoria	QA e bilanciamento ingestibili	Tassonomia, limiti agli effetti e test combinatori
UI sovraccarica	Intenti illeggibili	Filtri, priorità, trasparenza e modalità focus
Pathfinding lento	Preview scattosa	Chunk, cache, query asincrone e profiling
Divergenze di simulazione	Risultati differenti	Server authority, fixed-point, replay test
Leak di intenti	Vantaggio competitivo grave	Team relay, test pacchetti e audit
Blueprint spaghetti	Manutenzione difficile	API C++, Blueprint piccoli e data-driven
GAS troppo accoppiato	Simulatore dipendente da Actor runtime	Adapter tra GAS e stato tattico
Personaggi senza identità	Build intercambiabili	Kit fondamentale non sostituibile
Progressione pay/grind-to-win	Perdita di fiducia	Solo opzioni orizzontali e cosmetiche
Mod incompatibili	Crash e frammentazione	Schema, hash, sandbox e migration
Scope eccessivo	Progetto non completato	Vertical slice con gate rigoroso
Server costoso	Sostenibilità ridotta	Simulazione turn-based efficiente e autoscaling
Bassa popolazione	Matchmaking lento	Bot, custom lobby e modalità asincrone future

Strategia di testing.

Test	Descrizione
Unit test pathfinding	Percorsi, costi, blocchi, dislivelli
Golden test	Snapshot noto produce TurnLog noto
Replay determinism	Stesso log riprodotto più volte
Network privacy	Client nemico non riceve intenti
Packet loss	Preview e commit con latenza/perdita
Map validation	Spawn, obiettivi, connessioni e loop
Ability validation	Costi, tag, target e cooldown
Combinatorial test	Interazioni tra superfici ed effetti

Test	Descrizione
Soak test	Match ripetuti per ore
Mod schema test	File validi, invalidi e versioni precedenti
UX playtest	Comprensione senza spiegazione verbale
Accessibility	Contrasto, dimensione, input e motion

Definition of done per feature.

Una feature è completata soltanto quando:

- possiede requisiti e criteri di accettazione;
- funziona su server e client quando applicabile;
- non espone informazioni non autorizzate;
- dispone di log e debug visualization;
- possiede almeno un test automatico per la regola principale;
- è utilizzabile con input rimappato;
- produce messaggi di errore leggibili;
- è documentata per designer;
- non introduce riferimenti hard-coded ai contenuti;
- è stata provata in una build packaged, non soltanto in Play In Editor.

Decisioni da congelare dopo il vertical slice.

Decisione	Momento di lock
Dimensione squadra principale	Fine vertical slice
Sequenza di risoluzione	Prima dell'MVP
Modello delle celle	Prima del vertical slice
API pathfinding	Prima del vertical slice
Struttura degli intenti	Prima del prototipo di rete
Tassonomia Gameplay Tags	Versione iniziale prima dell'MVP
Schema dei contenuti	Alpha
Formato mod pubblico	Beta
Catalogo competitivo	Ogni stagione

Criterio finale di successo. RefactorTactics è riuscito quando i giocatori possono osservare una situazione multilivello, proporre un piano visibile ai compagni, modificare il terreno attraverso personaggi differenti e comprendere perché la risoluzione ha prodotto un determinato risultato. La quantità di personaggi, mappe e mod viene dopo questa qualità fondamentale.

- 1 10 19 **Balancing Blueprint and C++ | Unreal Engine 4.27 ...**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/balancing-blueprint-and-cplusplus?](https://dev.epicgames.com/documentation/en-us/unreal-engine/balancing-blueprint-and-cplusplus?application_version=4.27&utm_source=chatgpt.com)
[application_version=4.27&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/balancing-blueprint-and-cplusplus?application_version=4.27&utm_source=chatgpt.com)
- 2 12 **Gameplay Ability System | Unreal Engine 4.27 ...**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-ability-system?](https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-ability-system?application_version=4.27&utm_source=chatgpt.com)
[application_version=4.27&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-ability-system?application_version=4.27&utm_source=chatgpt.com)
- 3 **unreal.Actor — Unreal Python 5.7 (Experimental) ...**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/Actor?](https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/Actor?application_version=5.7&utm_source=chatgpt.com)
[application_version=5.7&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/Actor?application_version=5.7&utm_source=chatgpt.com)
- 4 20 **Gameplay Tags | Unreal Engine 4.27 Documentation**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-tags?](https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-tags?application_version=4.27&utm_source=chatgpt.com)
[application_version=4.27&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-tags?application_version=4.27&utm_source=chatgpt.com)
- 5 24 **A Formal Basis for the Heuristic Determination of Minimum ...**
https://www.cs.auckland.ac.nz/courses/compsci709s2c/resources/Mike.d/astarNilsson.pdf?utm_source=chatgpt.com
- 6 23 **FIndexArray | Unreal Engine 5.5 Documentation**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Runtime/AIModule/FGraphAStar/FIndexArray?](https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Runtime/AIModule/FGraphAStar/FIndexArray?application_version=5.5&utm_source=chatgpt.com)
[application_version=5.5&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Runtime/AIModule/FGraphAStar/FIndexArray?application_version=5.5&utm_source=chatgpt.com)
- 7 **Unreal Python 5.0 (Experimental) documentation**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/PredictProjectilePathParams?](https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/PredictProjectilePathParams?application_version=5.0&utm_source=chatgpt.com)
[application_version=5.0&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/PredictProjectilePathParams?application_version=5.0&utm_source=chatgpt.com)
- 8 21 **Input | Unreal Engine 4.27 Documentation**
https://dev.epicgames.com/documentation/en-us/unreal-engine/input?application_version=4.27&utm_source=chatgpt.com
- 9 **Unreal Engine 5.3 Release Notes - Epic Games Developers**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5.3-release-notes?](https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5.3-release-notes?application_version=5.3&utm_source=chatgpt.com)
[application_version=5.3&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5.3-release-notes?application_version=5.3&utm_source=chatgpt.com)
- 11 17 **Unreal Engine 4 For Unity Developers**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-4-for-unity-developers?](https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-4-for-unity-developers?application_version=4.27&utm_source=chatgpt.com)
[application_version=4.27&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-4-for-unity-developers?application_version=4.27&utm_source=chatgpt.com)
- 13 **Property Replication in Unreal Engine**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/property-replication-in-unreal-engine?](https://dev.epicgames.com/documentation/en-us/unreal-engine/property-replication-in-unreal-engine?application_version=5.2&utm_source=chatgpt.com)
[application_version=5.2&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/property-replication-in-unreal-engine?application_version=5.2&utm_source=chatgpt.com)
- 14 **Assets and Packages | Unreal Engine 4.27 Documentation**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/assets-and-packages?](https://dev.epicgames.com/documentation/en-us/unreal-engine/assets-and-packages?application_version=4.27&utm_source=chatgpt.com)
[application_version=4.27&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/assets-and-packages?application_version=4.27&utm_source=chatgpt.com)
- 15 **Unreal Engine 5.2 Release Notes**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5.2-release-notes?](https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5.2-release-notes?application_version=5.2&utm_source=chatgpt.com)
[application_version=5.2&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5.2-release-notes?application_version=5.2&utm_source=chatgpt.com)
- 16 **Non-Games Custom License Getting Started Guide**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/non-games-custom-license-unreal-engine-getting-started-](https://dev.epicgames.com/documentation/en-us/unreal-engine/non-games-custom-license-unreal-engine-getting-started-guide?application_version=4.27&utm_source=chatgpt.com)
[guide?application_version=4.27&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/non-games-custom-license-unreal-engine-getting-started-guide?application_version=4.27&utm_source=chatgpt.com)
- 18 **Blueprint Overview | Unreal Engine 4.27 Documentation**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-overview?](https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-overview?application_version=4.27&utm_source=chatgpt.com)
[application_version=4.27&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-overview?application_version=4.27&utm_source=chatgpt.com)
- 22 **UMG UI Designer Quick Start Guide - Epic Games Developers**
[https://dev.epicgames.com/documentation/en-us/unreal-engine/umg-ui-designer-quick-start-guide?](https://dev.epicgames.com/documentation/en-us/unreal-engine/umg-ui-designer-quick-start-guide?application_version=4.27&utm_source=chatgpt.com)
[application_version=4.27&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/umg-ui-designer-quick-start-guide?application_version=4.27&utm_source=chatgpt.com)

25 **Near Optimal Hierarchical Path-Finding**

https://webdocs.cs.ualberta.ca/~mmueller/ps/2004/hpastar.pdf?utm_source=chatgpt.com

26 **D* Lite**

https://aaai.org/Papers/AAAI/2002/AAAI02-072.pdf?utm_source=chatgpt.com

27 **Industry-standard navigation-mesh toolset for games · GitHub**

https://github.com/recastnavigation/recastnavigation?utm_source=chatgpt.com

28 **The EOS Online Subsystem (OSS) Plugin | Unreal Engine ...**

https://dev.epicgames.com/documentation/en-us/unreal-engine/the-eos-online-subsystem-oss-plugin?application_version=4.27&utm_source=chatgpt.com